

MatrixDSL

A Domain-Specific Language and LLVM-Based Compiler
for Matrix and AI Workloads

Project ID	A3
Team Number	13
Review	Review 1 — Initial Design, Planning and Feasibility
Maximum Marks	10
Review Date	_____
Faculty / Evaluator	_____

TEAM MEMBERS

Name	Register Number	Primary Responsibility
Vinay A	24BCB0131	Lexer, Parser, AST
Mudpe Parth Tulsidas	24BDS0353	Semantic Analysis, Symbol Table, Matrix Type System
Kandi Jeevitesh Reddy	24BCE0350	LLVM IR Generation, Optimization
Shreyas Mandem	24BCE0381	MDT ISA, Custom LLVM Backend

Repository · github.com/shreyasmandem/MatrixDSL

1. Abstract

Modern artificial intelligence and scientific-computing workloads rely heavily on matrix operations such as matrix multiplication, matrix addition, transposition, and element-wise activation functions. General-purpose programming languages can express these computations, but they do not directly expose domain-specific matrix semantics or provide a dedicated compilation path for a matrix-oriented target. This project presents MatrixDSL, a domain-specific language and LLVM-based compiler designed to provide a concise programming model for matrix and basic AI workloads while demonstrating target-specific code generation.

The compiler follows a multi-stage architecture consisting of lexical analysis, parsing, abstract syntax tree construction, semantic analysis, LLVM Intermediate Representation generation, LLVM optimization, and custom target code generation. The semantic analyzer performs compile-time validation of matrix dimensions and operation compatibility, allowing invalid matrix computations to be detected before code generation.

The project also defines a custom LLVM target named the MatrixDSL Target (MDT). MDT is intentionally small and contains three register and instruction domains: scalar, vector, and matrix. The target uses R0–R15 general-purpose registers, V0–V7 vector registers, and M0–M7 matrix registers. Its matrix operations are based on fixed 4 by 4 FP32 tiles. The custom LLVM backend describes these resources using LLVM TableGen and implements target registration, register classes, instruction selection and lowering, and assembly emission. Matrices larger than the hardware tile are handled through compiler-level tiling and decomposition.

The expected outcome is a functional end-to-end compiler capable of transforming high-level MatrixDSL programs into LLVM IR and subsequently into MDT assembly instructions, verified by an accompanying instruction-set simulator. The project evaluates correctness of generated code, quality of matrix dimension diagnostics, optimization behaviour, and generated instruction sequences using representative matrix and AI workloads.

2. Problem Statement and Motivation

2.1 Context

Matrix multiplication is the dominant computational primitive in machine learning. Training and inference for neural networks reduce almost entirely to sequences of matrix products interleaved with element-wise activation functions. This has driven the design of specialised hardware — NVIDIA tensor cores, Google's Tensor Processing Unit, Apple's Neural Engine — in which the fundamental compute unit operates on a fixed-size matrix tile rather than on individual scalars.

Specialised hardware creates a compiler problem. A processor that offers a matrix multiply instruction is only useful if a compiler can target it, and a general-purpose language gives the compiler very little to work with.

2.2 The problem

When a matrix multiplication is written in a general-purpose language, it appears as a triple-nested loop over scalar operations:

```

for (int i = 0; i < N; i++)
  for (int j = 0; j < N; j++)
    for (int k = 0; k < N; k++)
      C[i][j] += A[i][k] * B[k][j];

```

The programmer knows this is a matrix multiplication. The compiler does not. To target a matrix instruction, the compiler must *recover* that knowledge by recognising the loop idiom — a fragile analysis that is easily defeated by loop reordering, bounds that are not compile-time constants, aliasing between the operands, or any of the optimizer's own earlier transformations.

Two consequences follow:

1. **Semantic information is discarded.** Matrix shape is present in the programmer's intent but absent from the program text, so dimension errors surface as incorrect results or memory corruption at runtime rather than as compile-time errors.
2. **Target-specific instructions are hard to reach.** Mapping onto a matrix instruction depends on idiom recognition succeeding, which cannot be relied upon.

2.3 Motivation

MatrixDSL addresses both by making matrix operations first-class in the language. `matmul(A, B)` is a single AST node with known operand shapes. The compiler never has to recover intent, because the intent was never lost. Shape becomes part of the type system, so dimension errors are compile-time errors. Mapping onto a target matrix instruction becomes a direct translation rather than a pattern-matching gamble.

This is the same architectural bet made by production systems. OpenAI's Triton compiles a Python-like tile-level DSL through an LLVM-based IR to GPU code, precisely so that block-level semantics reach the code generator intact. Halide separates the algorithm from its schedule and lowers through LLVM to target CPUs, GPUs and DSPs from a single description. MatrixDSL demonstrates the same principle at a scale appropriate to a course project, and takes it one step further by defining the target architecture as well as the language.

2.4 Beneficiaries

Beneficiary	Benefit
Compiler-design students	A complete, readable example of a custom LLVM target
DSL designers	A worked case of shape-aware type checking driving code generation
Hardware architects	A demonstration of how a new matrix ISA is made reachable by a compiler
The project team	Practical experience across the full compilation pipeline

3. Objectives, Scope and Limitations

3.1 Measurable objectives

#	Objective	Success criterion
O1	Design and implement a lexer and recursive-descent parser for MatrixDSL	100% of grammar productions parsed; 13 parser tests passing
O2	Implement a shape-aware semantic analyzer	All 9 invalid-shape cases rejected with correct diagnostics; 25 semantic tests passing
O3	Generate valid, verifiable LLVM IR for all 8 language operations	<code>llvm::verifyModule</code> clean for every test program
O4	Define and implement the MDT target as a custom LLVM backend	All 17 MDT instructions selectable; correct assembly emitted
O5	Demonstrate 4 by 4 tiling for matrices up to 128 by 128	16x16, 32x32 and 128x128 multiplies produce correct results
O6	Verify end-to-end correctness by differential testing	MDT simulator output matches native x86-64 output on all integration tests

3.2 In scope

- MatrixDSL v1.0: declarations, literals, addition, subtraction, scalar multiply, matrix multiply, transpose, ReLU, print
- FP32 element type; matrices from 1x1 to 256x256
- Complete frontend: lexer, parser, AST, semantic analysis with shape checking
- LLVM IR generation and use of LLVM's existing optimization pipeline
- MDT target: 3 register classes, 17 instructions, TableGen description, backend
- Compiler-level 4 by 4 tiling
- MDT instruction-set simulator for verification
- Test suite covering valid, invalid and boundary cases

3.3 Out of scope

Excluded	Reason
Physical processor or FPGA implementation	Hardware implementation is a separate discipline
Cache hierarchy, pipelining, hazard modelling	MDT is a compilation target, not a microarchitecture study
GPU execution model, full TPU systolic array	Beyond the scope of a single-semester project
Source-level control flow, user functions	Not required to demonstrate the compilation path
Matrix inverse, determinant, decompositions	Adds numerical complexity without compiler-design value
Convolution, pooling, softmax, autodiff	Natural extensions, recorded as future work
Dynamic matrix shapes	Compile-time shape knowledge is central to the design
Binary instruction encoder	Readable assembly is the required backend output

3.4 Assumptions and limitations

- Matrix dimensions are compile-time constants, known statically.
 - All arithmetic is FP32; no mixed precision, no numerical-stability analysis.
 - The project does not claim MDT is faster than a real CPU. Without silicon or a cycle-accurate model, such a comparison would be unfounded. The contribution is the compiler and backend design, and the correctness of generated target code.
 - The simulator is functional, not cycle-accurate.
-

4. Background Study and Technical Understanding

4.1 Relevant compiler concepts

Concept	Application in this project
Lexical analysis	Hand-written scanner producing a typed token stream
Recursive-descent parsing	LL(1) grammar after left-recursion elimination
Abstract syntax trees	Typed node hierarchy shared across all four modules
Symbol tables	Name-to-shape binding with scope management
Type checking	Shape checking as type checking; dimensions are part of the type
Intermediate representation	LLVM IR as the frontend/backend interface
Optimization passes	LLVM's existing pipeline; constant folding, DCE, CSE, loop simplification
Instruction selection	SelectionDAG pattern matching, TableGen-described
Register allocation	Three disjoint classes; matrix spilling at 64 bytes per tile
Code generation	Machinelnstr to MCInst to assembly text
Loop tiling	Decomposition of large matrices into fixed hardware tiles

4.2 Survey of existing approaches

System	Domain	IR strategy	Target	Relevance
Triton (OpenAI)	GPU tile programming	Triton-IR on MLIR, lowered to LLVM IR	PTX, AMDGCN	Direct precedent: tile-level DSL reaching a matrix-capable target through LLVM
Halide	Image processing	Custom IR, lowered to LLVM IR	x86, ARM, CUDA, Hexagon	Separation of algorithm from schedule; retargetability via LLVM
TVM	Deep learning	Relay and TIR	Many, incl. accelerators	Tensor-level DSL with explicit hardware-tile mapping
MLIR	Compiler infrastructure	Multi-level dialects	Various	Progressive lowering from domain semantics to machine code
Rust	Systems programming	LLVM IR with <code>noalias</code>	Native	Frontend proofs communicated to the optimizer as IR facts
Cpu0	Teaching backend	LLVM IR	Cpu0 (fictional)	Reference for constructing a new LLVM target from scratch

4.3 Identified gap

Production systems (Triton, TVM, Halide) target real hardware and are consequently very large, with the domain-to-target path spread across multiple IR levels and tens of thousands of lines. Teaching resources (the

Kaleidoscope tutorial, the Cpu0 backend) demonstrate either a frontend reaching LLVM IR or a custom backend from LLVM IR, but not a complete path in which a *domain-specific* frontend drives a *domain-specific* target with matching first-class matrix operations.

MatrixDSL occupies that gap: a compilation path small enough to read end to end, in which the matrix abstraction is preserved unbroken from source syntax through the type system, through LLVM IR, and into a matrix instruction in the target ISA — with the target's matrix registers and the language's matrix type deliberately co-designed.

4.4 Justification of technology choices

Choice	Alternatives considered	Justification
LLVM	Custom IR and code generator; GCC	Mature optimizer obtained without reimplementation; documented backend interface; genuine retargetability, which enables differential testing against the host
C++17	Python, Rust	LLVM's primary API is C++; matches course expectations
Hand-written lexer/parser	Flex/Bison, ANTLR	Grammar is small and LL(1); hand-writing gives better diagnostics and clearer demonstration of the concepts being assessed
Recursive descent	LR, LALR	Directly mirrors the grammar; straightforward to debug
TableGen backend	Direct IR-to-assembly emission	The canonical LLVM approach and the strongest demonstration; a direct emitter is retained as a documented fallback (Section 5.4)
4 by 4 FP32 tile	8x8, 16x16, scalar-only	64 bytes per register; meaningful work per instruction while remaining simulator-friendly
Custom simulator	Native execution, QEMU, gem5	MDT does not exist; a functional simulator is the only way to execute and verify generated code

5. Requirements and Feasibility

5.1 Software requirements

Component	Version	Purpose
C++ compiler	GCC 11+ / Clang 14+ / MSVC 2022	C++17 support
LLVM	17 or 18 (pinned)	IR APIs, optimizer, backend infrastructure, TableGen
CMake	3.16+	Build system
Git	2.30+	Version control
CTest	bundled with CMake	Test driving

The entire toolchain is C++ and CMake. There is no scripting-language dependency: the test runner, the reference implementations and the MDT simulator are all C++ programs built by the same CMake configuration as the

compiler itself.

All four members build against the same pinned LLVM version to avoid API drift between releases.

5.2 Hardware requirements

Resource	Minimum	Recommended	Note
CPU	4 cores	8 cores	LLVM builds parallelise well
RAM	8 GB	16 GB	LLVM linking is memory-hungry
Disk	30 GB	50 GB	LLVM source and build tree
OS	Windows 11, Linux, macOS	Linux or WSL2	LLVM tooling is best supported on Linux

No GPU or specialised hardware is required. MDT is simulated.

5.3 Test inputs and datasets

The project requires no external dataset. Test inputs are MatrixDSL programs written by the team:

Set	Count	Purpose
Valid programs	25+	Correct compilation and execution
Invalid programs	15+	Diagnostic correctness
Boundary programs	12+	Dimension limits, non-multiples of 4
Integration programs	9	Full pipeline including tiling
Benchmarks	5	4x4 through 128x128 workloads

Reference results are produced by a straightforward C++ implementation of each operation, so no hand-computed expected values are required.

5.4 Risk analysis

#	Risk	Likelihood	Impact	Mitigation
R1	LLVM backend complexity exceeds available time	High	High	Phased implementation (scalar, then vector, then matrix), each independently demonstrable; direct IR-to-assembly emitter as documented fallback; decision point end of Week 8
R2	LLVM build/setup difficulty on Windows	Medium	Medium	Use prebuilt LLVM via vcpkg or WSL2; pin one version across the team; document setup in the README
R3	Serial dependency — three modules blocked on the parser	High	High	AST and token interfaces frozen on <code>main</code> before implementation; hand-constructed AST fixtures let downstream modules develop and test in parallel
R4	Interface drift between IR generation and backend	Medium	High	Matrix intrinsic naming (<code>@mmt.matmul.4x4</code>) agreed and documented before either side starts
R5	Tiling correctness for non-multiples of 4	Medium	Medium	Zero-padding strategy specified up front; dedicated boundary tests
R6	Register pressure with only 8 matrix registers	Low	Medium	Spill path specified; tiling loop written to keep at most 3 tiles live
R7	Member unavailability	Low	Medium	Interfaces are documented, so work is transferable; every module has a named secondary

R1 and R3 are the two risks that can actually sink the project, and both have concrete, already-executed mitigations: the fallback backend path is designed in advance rather than improvised, and the shared interfaces are committed to `main` as part of this review rather than deferred.

5.5 Feasibility assessment

Dimension	Assessment
Technical	Feasible. Every component has established precedent. The backend is the demanding part and carries a designed fallback.
Schedule	Feasible in 12 weeks with the phased plan in Section 10. Backend phases are ordered so that partial completion still yields a working artifact.
Resource	Feasible. Standard development machines; no licensing or hardware cost.
Skill	Feasible. Frontend work is standard course material. Backend work is new to the team and is therefore allocated the longest continuous block and the earliest start.

6. Proposed Methodology

6.1 Approach

1. **Freeze the interfaces first.** Token enumeration, AST node hierarchy, `MatrixType` and `SymbolTable` signatures, and the matrix intrinsic naming are agreed and committed before implementation. This converts a serial pipeline into four parallel workstreams.
2. **Freeze the ISA before the backend.** The 17 instructions, three register classes and assembly syntax are fixed so that backend work and simulator work proceed together against one specification.
3. **Build the frontend bottom-up.** Lexer, then parser, then AST, then semantic analysis — each with its own tests before the next begins.
4. **Reach LLVM IR early.** Generating verifiable IR for even a single operation validates the entire frontend contract.
5. **Build the backend in phases.** Target registration, scalar, memory, control flow, vector, matrix, tiling. Each phase is independently demonstrable.
6. **Verify by differential testing.** Compile the same program to the host and to MDT; the results must agree.

6.2 Key technical decisions

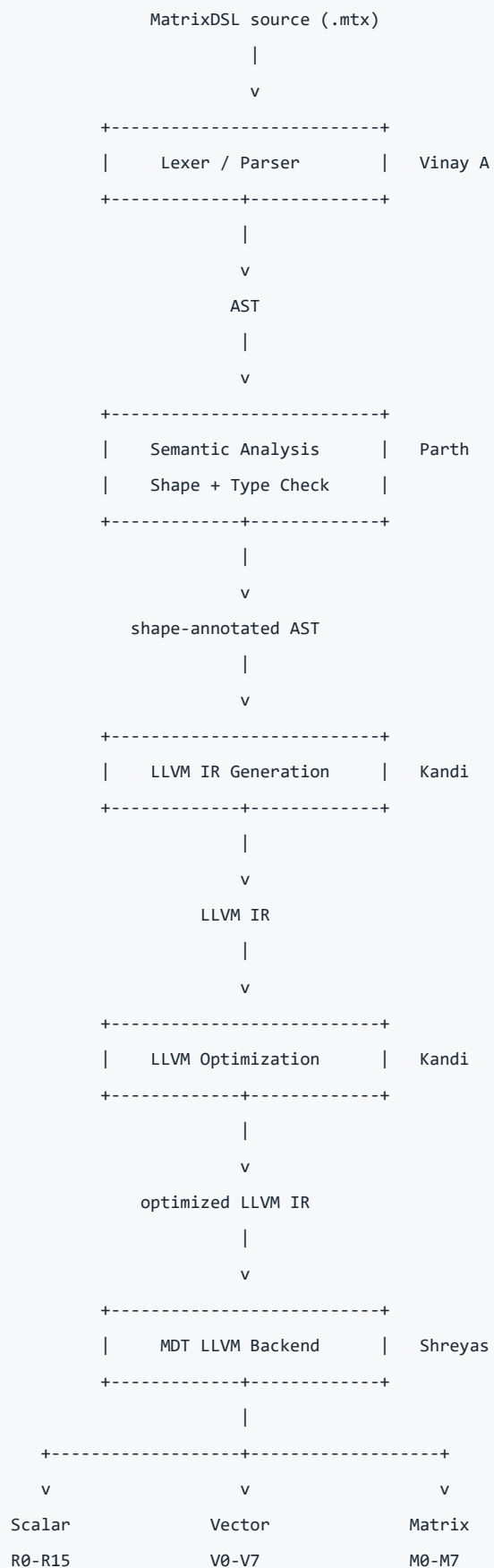
Decision	Choice	Rationale
Matrix representation in IR	Descriptor <code>{float*, i32, i32}</code>	Separates data from shape; supports both static checking and runtime printing
Storage order	Row-major	Matches C convention; makes rows contiguous, simplifying tile extraction
Matrix operations in IR	Reserved intrinsic calls (<code>@mdt.matmul.4x4</code>)	Avoids fragile loop-idiom recognition; survives the optimizer intact
Tile size in intrinsic name	Encoded (<code>4x4</code>)	The backend never infers tile geometry; tiling has already run
Tile type in TableGen	<code>v16f32</code>	Reuses LLVM vector machinery; avoids patching LLVM core with a new MVT
Non-multiple-of-4 dimensions	Zero-pad to next multiple	Simple and correct; logical shape preserved in the descriptor
Verification strategy	Differential against host	Removes the need for hand-computed expected values

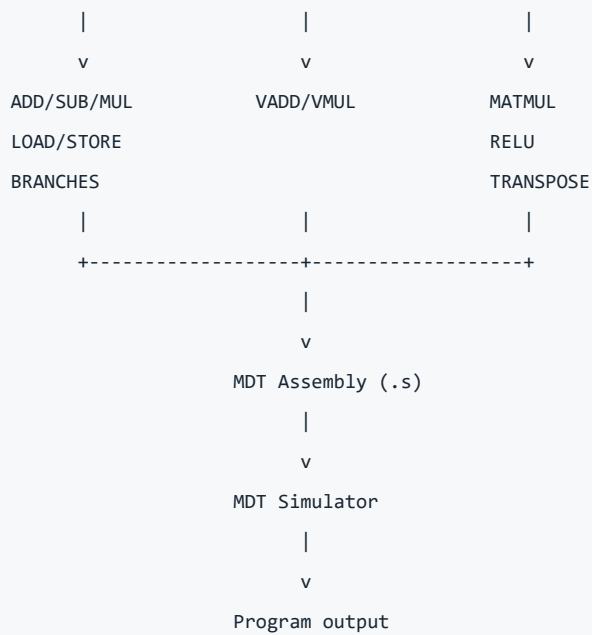
6.3 Development practices

- Feature branches per member; `main` must always build and pass the full suite.
 - Every module ships with its tests; a module is not complete until its tests pass.
 - Interface changes require agreement from both sides of the interface.
 - Weekly integration checkpoint.
-

7. Architecture and Data Flow

7.1 System architecture





7.2 Data-flow example

Source statement `C = matmul(A, B);` where A and B are 4x4:

Stage	Representation
Tokens	<code>IDENT(C) ASSIGN MATMUL LPAREN IDENT(A) COMMA IDENT(B) RPAREN SEMI</code>
AST	<code>Assignment(VariableExpr(C), CallExpr(matmul, [Var(A), Var(B)]))</code>
After sema	Same tree; <code>CallExpr.resultType = 4x4</code> , checked against declared <code>C</code>
LLVM IR	<code>call void @mdt.matmul.4x4(float* %C.data, float* %A.data, float* %B.data)</code>
After opt	Unchanged — a single call with no foldable operands
MDT assembly	<code>LOAD M1, [R1+0] / LOAD M2, [R2+0] / MATMUL M0, M1, M2 / STORE M0, [R0+0]</code>

7.3 Module interaction



The frontend carries no LLVM dependency, so two of four members can work productively before LLVM is installed. The simulator is likewise standalone.

8. Module Description

8.1 Lexer

Field	Value
Purpose	Convert source text into a typed token stream
Input	MatrixDSL source (<code>.mtx</code>)
Output	<code>std::vector<Token></code> with line and column information
Algorithm	Single-pass scanner with one-character lookahead; maximal-munch identifiers and numbers; keyword recognition by table lookup after identifier scan
Owner	Vinay A

8.2 Parser

Field	Value
Purpose	Verify grammatical structure and construct the AST
Input	Token stream
Output	AST rooted at <code>Program</code>
Algorithm	Recursive descent, LL(1), one token of lookahead; precedence climbing for binary operators; panic-mode recovery at statement boundaries
Owner	Vinay A

8.3 Semantic Analyzer and Symbol Table

Field	Value
Purpose	Validate names and matrix shapes; annotate the AST with result types
Input	AST
Output	Shape-annotated AST, populated symbol table, diagnostics
Algorithm	Post-order traversal computing each expression's result shape bottom-up; symbol table as a scoped hash map; shape rules per Section 5 of the language spec
Owner	Mudpe Parth Tulsidas

8.4 LLVM IR Generator

Field	Value
Purpose	Lower the annotated AST to LLVM IR
Input	Shape-annotated AST
Output	<code>llvm::Module</code>
Algorithm	Recursive AST walk using <code>IRBuilder</code> ; matrices allocated as FP32 arrays with descriptors; matrix operations emitted as reserved intrinsic calls
Owner	Kandi Jeevitesh Reddy

8.5 Optimizer

Field	Value
Purpose	Apply LLVM's optimization pipeline
Input	<code>llvm::Module</code>
Output	Optimized <code>llvm::Module</code>
Algorithm	<code>PassBuilder</code> at <code>-O2</code> ; constant folding, DCE, CSE, loop simplification; matrix intrinsics marked to survive as opaque calls
Owner	Kandi Jeevitesh Reddy

8.6 MDT Backend

Field	Value
Purpose	Translate optimized LLVM IR into MDT assembly
Input	Optimized <code>llvm::Module</code>
Output	MDT assembly (<code>.s</code>)
Algorithm	TableGen-described registers and instructions; SelectionDAG instruction selection; custom SDNodes for matrix operations; tiling pass before selection; register allocation across three disjoint classes
Owner	Shreyas Mandem

8.7 MDT Simulator

Field	Value
Purpose	Execute MDT assembly to verify generated code
Input	MDT assembly (<code>.s</code>)
Output	Program output, final register and memory state
Algorithm	Two-pass assembly (label resolution, then decode); fetch-decode-execute loop over the 17-instruction ISA; functional, not cycle-accurate
Owner	Shreyas Mandem

8.8 Runtime

Field	Value
Purpose	Matrix allocation and formatted output
Input	Matrix descriptors
Output	Allocated storage; printed matrices
Algorithm	64-byte aligned allocation so 4x4 tiles never straddle an alignment boundary; row-major formatted printing
Owner	Kandi Jeevitesh Reddy

9. Team Responsibility Matrix

Member	Primary Responsibility	Supporting Responsibility	Review 1 Evidence	Review 2 Target	Review 3 Target
Vinay A 24BCB0131	Lexer, Parser, AST	Frontend integration; grammar documentation	EBNF grammar; token specification; AST hierarchy design; <code>Lexer.h</code> / <code>AST.h</code> interfaces	Working lexer and parser producing AST for all constructs; 23 frontend tests passing	Frontend integrated, error recovery, full diagnostics
Mudpe Parth Tulsidas 24BDS0353	Semantic Analysis, Symbol Table, Matrix Type System	Negative and shape test suites	Shape rule specification; <code>MatrixType</code> / <code>SymbolTable</code> interfaces; 25-case semantic test plan	Shape checking operational; all invalid programs correctly rejected	Full diagnostics with source locations; semantic tests complete
Kandi Jeevitesh Reddy 24BCE0350	LLVM IR Generation, Optimization	Runtime library; integration	Matrix IR representation design; intrinsic naming contract; optimization pass plan	Valid IR for all 8 operations; <code>-O2</code> pipeline running	Optimized IR, before/after analysis, runtime complete
Shreyas Mandem 24BCE0381	MDT ISA, Custom LLVM Backend	Backend tests; assembly verification	Frozen MDT ISA specification; TableGen register and instruction descriptions; working MDT simulator; backend test suite	Target registered; scalar and memory instructions selecting correctly	Vector and matrix instructions; tiling; end-to-end assembly

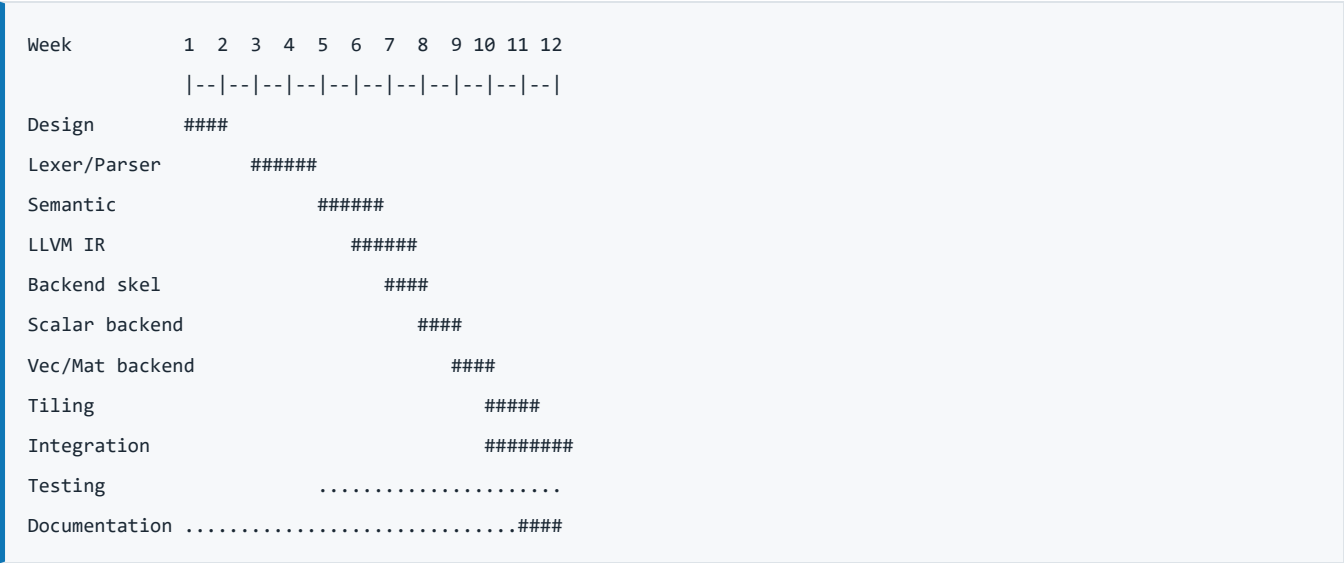
Every member owns at least one technical component and supports at least one integration, testing or documentation activity.

10. Timeline and Review-Wise Deliverables

10.1 Twelve-week plan

Weeks	Focus	Deliverable	Lead
1–2	Language and MDT design	Grammar, type rules, ISA, register model, instruction semantics	All
3–4	Lexer and parser	Tokens, recursive-descent parser, AST	Vinay
5–6	Semantic analysis	Symbol table, matrix type system, shape diagnostics	Parth
6–7	LLVM IR	AST to LLVM IR, matrix memory representation	Kandi
7–8	MDT backend skeleton	Target registration, registers, TableGen	Shreyas
8–9	Scalar backend	ADD/SUB/MUL, LOAD/STORE, branches	Shreyas
9–10	Vector and matrix backend	VADD/VMUL, MATMUL/RELU/TRANPOSE	Shreyas
10–11	Tiling and integration	4x4 tiling, end-to-end compilation	All
12	Testing and report	Benchmarks, final demonstration, documentation	All

10.2 Gantt chart



denotes primary activity, . denotes continuous activity.

10.3 Review-wise deliverables

Review	Deliverables
Review 1	Problem definition, objectives, background survey, architecture, module design, responsibility matrix, timeline, risk register, frozen grammar and ISA, shared interfaces committed, MDT simulator prototype, test plan
Review 2	Working lexer, parser, AST; semantic analysis with shape checking; LLVM IR generation for core operations; MDT target registered with scalar instructions selecting; frontend and semantic test suites passing
Review 3	Complete pipeline: MatrixDSL to LLVM IR to optimized IR to MDT assembly; vector and matrix instructions; 4x4 tiling for large matrices; differential verification against host; benchmark results; final report and demonstration

10.4 Integration plan

Integration is continuous rather than deferred to the end:

Milestone	Integration event
End Week 4	Parser output consumed by a stub semantic analyzer
End Week 6	Semantic analyzer output consumed by IR generation
End Week 7	First end-to-end run: source to LLVM IR to host executable
End Week 9	First MDT assembly generated and executed in the simulator
End Week 11	Full pipeline including tiling

The Week 7 milestone matters most: it exercises every frontend interface before the backend is finished, so interface defects surface while there is still time.

11. Initial Progress

11.1 Repository

<https://github.com/shreyasmandem/MatrixDSL>

Complete folder structure established with documentation, shared interface headers, example programs, test cases, build system and helper scripts. Per-member branches created for individual contributions.

11.2 Completed for Review 1

Item	Status	Evidence
MatrixDSL grammar formalised in EBNF	Complete	<code>docs/grammar.md</code>
Language specification with shape rules	Complete	<code>docs/language-spec.md</code>
System architecture and data flow	Complete	<code>docs/architecture.md</code>
MDT ISA frozen	Complete	<code>docs/isa.md</code>
Backend design with risk mitigation	Complete	<code>docs/llvm-backend.md</code>
Testing strategy	Complete	<code>docs/testing.md</code>
Shared interfaces (tokens, AST, types)	Complete	<code>compiler/frontend/**/*.h</code>
Build system	Complete	<code>CMakeLists.txt</code> , <code>scripts/</code>
Example programs	Complete	<code>examples/*.mtx</code>
Test cases (valid/invalid/boundary)	Complete	<code>tests/</code>
MDT instruction-set simulator	Working prototype	<code>tools/mdtsim/</code>

11.3 Verifiable technical output

The MDT simulator is a working executable that assembles and runs MDT assembly, implementing all 17 instructions. It demonstrates that the ISA specification is complete and unambiguous, and it provides the verification mechanism the backend will be tested against.

Sample session:

```
$ ./mdtsim tests/backend/matmul_4x4.s --dump-matrix M0
```

```
MDT Simulator v1.0
```

```
Loaded 12 instructions.
```

```
M0 =
```

```
[ 30.00  70.00 110.00 150.00 ]
[ 70.00 174.00 278.00 382.00 ]
[ 110.00 278.00 446.00 614.00 ]
[ 150.00 382.00 614.00 846.00 ]
```

```
Executed 12 instructions in 12 cycles.
```

This is a genuine proof of start: the target architecture is not merely specified on paper, it is executable.

12. Testing Strategy

Summarised here; full inventory in [docs/testing.md](#).

Stage	Valid	Invalid	Boundary	Total
Lexer	6	2	2	10
Parser	10	2	1	13
Semantic	7	9	9	25
LLVM IR	9	0	0	9
Backend	12	0	0	12
Integration	6	0	3	9
Total	50	13	15	78

Representative cases

Valid. 4x4 identity multiply; `matmul(A[4][3], B[3][5])` yielding 4x5; chained `relu(matmul(A,B))`; 16x16 and 32x32 tiled multiplication.

Invalid. `matmul(A[4][3], B[7][5])` — inner dimensions disagree; `A[4][4] + B[3][3]` — shapes differ; assignment of a 4x5 result to a 4x4 target; undeclared identifier; redeclaration; ragged matrix literal.

Boundary. 1x1 minimum; 256x256 maximum; 257x257 rejected; zero and negative dimensions rejected; 5x7 non-multiple-of-4 requiring zero-padded tiling; 1x4 by 4x1 degenerate multiply.

Differential verification

The same MatrixDSL program is compiled to the host architecture and to MDT. The simulator result must match the native result element for element. This removes the need for hand-computed expected values and localises any divergence to the backend.

13. Expected Results and Extension Potential

13.1 Expected results

#	Expected outcome
1	A complete compiler translating MatrixDSL to MDT assembly
2	Compile-time detection of every dimension error in the invalid test set, with actionable diagnostics
3	Verifiable LLVM IR for all 8 language operations, before and after optimization
4	All 17 MDT instructions correctly selected and emitted
5	Correct 4x4 tiling for matrices up to 128x128
6	Simulator results matching native execution across the integration suite
7	Documented analysis of generated instruction sequences and optimizer effect

13.2 Evaluation

Evaluation focuses on compiler correctness and generated-code quality, not on claimed hardware speed:

- Correctness of results against a reference implementation
- Correctness and clarity of dimension diagnostics
- Generated LLVM IR before and after optimization
- Generated MDT instruction sequences, including instruction counts per operation
- Tiling behaviour across matrix sizes

The project explicitly does **not** claim MDT outperforms real processors. Without silicon or a cycle-accurate model such a claim would be unfounded.

13.3 Extension potential

Extension	Description
Additional AI operations	Convolution, pooling, softmax, sigmoid, tanh
Binary encoder	Emit 32-bit encoded instructions rather than assembly text
Cycle-accurate simulation	Add a pipeline and memory model for performance study
Auto-tuned tiling	Search tile schedules rather than fixing 4x4
Mixed precision	FP16 and BF16 tiles, as used in real accelerators
Operator fusion	Fuse <code>relu(matmul(A,B))</code> into a single fused instruction
MLIR dialect	Reimplement the frontend as an MLIR dialect for progressive lowering
Real hardware	FPGA implementation of the MDT matrix unit

Operator fusion is the most promising publication-adjacent direction. Because matrix operations survive to the backend as recognisable intrinsics rather than dissolving into loop nests, fusing `relu(matmul(A,B))` into one instruction is a direct pattern match — and its benefit can be measured as an instruction-count reduction against the unfused baseline.

14. References

1. Lattner, C. and Adve, V. *LLVM: A Compilation Framework for Lifelong Program Analysis and Transformation*. CGO, 2004.
 2. LLVM Project. *Writing an LLVM Backend*. <https://llvm.org/docs/WritingAnLLVMBackend.html>
 3. LLVM Project. *TableGen Overview*. <https://llvm.org/docs/TableGen/>
 4. LLVM Project. *The LLVM Target-Independent Code Generator*. <https://llvm.org/docs/CodeGenerator.html>
 5. LLVM Project. *LLVM Language Reference Manual*. <https://llvm.org/docs/LangRef.html>
 6. Tian, C. *Tutorial: Creating an LLVM Backend for the Cpu0 Architecture*. <http://jonathan2251.github.io/lbd/>
 7. Tillet, P., Kung, H.T. and Cox, D. *Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations*. MAPL, 2019.
 8. OpenAI. *Introducing Triton: Open-Source GPU Programming for Neural Networks*. <https://openai.com/index/triton/>
 9. Ragan-Kelley, J. et al. *Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines*. PLDI, 2013.
 10. Chen, T. et al. *TVM: An Automated End-to-End Optimizing Compiler for Deep Learning*. OSDI, 2018.
 11. Lattner, C. et al. *MLIR: Scaling Compiler Infrastructure for Domain Specific Computation*. CGO, 2021.
 12. Jouppi, N. et al. *In-Datacenter Performance Analysis of a Tensor Processing Unit*. ISCA, 2017.
 13. Kung, H.T. and Leiserson, C.E. *Systolic Arrays for VLSI*. Sparse Matrix Proceedings, 1979.
 14. Aho, A., Lam, M., Sethi, R. and Ullman, J. *Compilers: Principles, Techniques and Tools*. 2nd edition, Pearson, 2006.
 15. Appel, A. *Modern Compiler Implementation in C*. Cambridge University Press, 1998.
 16. Goto, K. and van de Geijn, R. *Anatomy of High-Performance Matrix Multiplication*. ACM TOMS, 2008.
-

Appendix A — Contribution Log

See `docs/review1/contribution-log.md`.

Appendix B — Repository Structure

See `README.md`.

Appendix C — Risk Register

See Section 5.4 and `docs/review1/risk-register.md`.